```
def is_variable(x):
    """
    Determines if an expression is a variable.
    By convention, variables are single characters or lowercase strings.
    """
    return isinstance(x, str) and x.islower()

def is_compound(x):
    """
    Determines if an expression is a compound term (e.g., function or predicate).
    Represented as a tuple: (op, arg1, arg2, ...)
    """
    return isinstance(x, tuple) and len(x) > 0

def occurs_check(var, term, subst):
    """
    Returns True if the variable 'var' occurs anywhere within the 'term',
    resolving any existing substitutions along the way.
    """
    if var == term:
        return True
    elif is_variable(term) and term in subst:
        return occurs_check(var, subst[term], subst)
    elif is_compound(term):
        # Scan through the operator arguments, omitting the operator name itself at index 0
        return any(occurs_check(var, arg, subst) for arg in term[1:])
    return False

def unify(x, y, subst=None):
    """
    Computes the Most General Unifier (MGU) of two FOL expressions x and y.
    Returns a dictionary of substitutions if successful, or None if unification fails.
    """
    if subst is None:
        subst = {}

    # If the substitution state is already broken, bubble up the failure
    if subst is None:
        return None

    # Case 1: Identical expressions
    if x == y:
        return subst

    # Case 2: x is a variable
    elif is_variable(x):
        return unify_var(x, y, subst)

    # Case 3: y is a variable
    elif is_variable(y):
        return unify_var(y, x, subst)

    # Case 4: Both are compound expressions (predicates or functions)
    elif is_compound(x) and is_compound(y):
        # Predicate/function symbols and argument lengths must match exactly
        if x[0] != y[0] or len(x) != len(y):
            return None

        # Recursively unify arguments sequentially, passing forward the updated substitutions
        for arg_x, arg_y in zip(x[1:], y[1:]):
            subst = unify(arg_x, arg_y, subst)
            if subst is None:
                return None
        return subst

    # Case 5: Constant mismatch or structural type mismatch
    else:
        return None

def unify_var(var, val, subst):
    """
    Helper function to handle variable bindings.
    """
    # If the variable already has an assigned value, unify its bound value with val
    if var in subst:
        return unify(subst[var], val, subst)

    # If val is a variable that already has an assigned value, unify var with its bound value
    elif is_variable(val) and val in subst:
        return unify(var, subst[val], subst)
```

```
    # Perform the occurs check to avoid infinite recursion loops (e.g., trying to unify x with f(x))
    elif occurs_check(var, val, subst):
        return None

    # Safe to extend the substitution set
    else:
        new_subst = subst.copy()
        new_subst[var] = val
        return new_subst

# ========================================
# Example Scenarios & Verification
# ========================================
if __name__ == "__main__":
    print("--- Running FOL Unification Tests ---")

    # Example 1: Basic matching
    # Predicate: knows(John, x) and knows(John, Elizabeth)
    # Target result: {'x': 'Elizabeth'}
    expr1 = ("knows", "John", "x")
    expr2 = ("knows", "John", "Elizabeth")
    print(f"Unifying {expr1} and {expr2}:")
    print("Result:", unify(expr1, expr2))
    print("-" * 40)

    # Example 2: Transitive variables
    # Predicate: P(x, y) and P(a, z)
    # Target result: {'x': 'a', 'y': 'z'}
    expr3 = ("P", "x", "y")
    expr4 = ("P", "a", "z")
    print(f"Unifying {expr3} and {expr4}:")
    print("Result:", unify(expr3, expr4))
    print("-" * 40)

    # Example 3: Nested function structures
    # Predicate: P(f(a), g(y)) and P(x, z)
    # Target result: {'x': ('f', 'a'), 'z': ('g', 'y')}
    expr5 = ("P", ("f", "a"), ("g", "y"))
    expr6 = ("P", "x", "z")
    print(f"Unifying {expr5} and {expr6}:")
    print("Result:", unify(expr5, expr6))
    print("-" * 40)

    # Example 4: Structurally bound conflict (Should Fail)
    # Predicate: P(f(a), g(y)) and P(x, x)
    # Why it fails: x binds to f(a), then second argument forces f(a) to match g(y), which fails.
    expr7 = ("P", ("f", "a"), ("g", "y"))
    expr8 = ("P", "x", "x")
    print(f"Unifying {expr7} and {expr8}:")
    print("Result:", unify(expr7, expr8))
    print("-" * 40)

    # Example 5: Occurs Check Trigger (Should Fail)
    # Predicate: P(x) and P(f(x))
    # Why it fails: A variable cannot be unified with a function containing itself.
    expr9 = ("knows(father(y),y)", "knows(x,x)")
    expr10 = ("P", ("f", "x"))
    print(f"Unifying {expr9} and {expr10}:")
    print("Result:", unify(expr9, expr10))
    print("-" * 40)
```

```
--- Running FOL Unification Tests ---
Unifying ('knows', 'John', 'x') and ('knows', 'John', 'Elizabeth'):
Result: {'x': 'Elizabeth'}
----------------------------------------
Unifying ('P', 'x', 'y') and ('P', 'a', 'z'):
Result: {'x': 'a', 'y': 'z'}
----------------------------------------
Unifying ('P', ('f', 'a'), ('g', 'y')) and ('P', 'x', 'z'):
Result: {'x': ('f', 'a'), 'z': ('g', 'y')}
----------------------------------------
Unifying ('P', ('f', 'a'), ('g', 'y')) and ('P', 'x', 'x'):
Result: None
----------------------------------------
Unifying ('knows(father(y),y)', 'knows(x,x)') and ('P', ('f', 'x')):
Result: None
----------------------------------------
```